CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

Worksheet No. - 06

Student Name: Sukhleen Kaur **UID: 25MCA20031**
Branch: MCA **Section/Group:25MCA-6B**
Semester: 3rd **Date of Performance: 16.10.2026**
Subject Name: Automation Software Testing **Subject Code: 25CAH-705**

Aim/Overview of the practical: (A). Develop and execute automated test cases using Playwright to validate the search functionality of a web application by verifying relevant results for different input conditions.

Objective: The objective is to develop and execute automated test cases using Playwright to validate the search functionality of a web application by checking the accuracy and relevance of search results for different input conditions, including valid, invalid, and empty inputs.

Code

```
import asyncio
import os
from playwright.async_api import async_playwright

async def test_search():
    os.makedirs("screenshots", exist_ok=True)

    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=False)
        page = await browser.new_page(
            viewport={"width": 1366, "height": 768}
        )

        url = "https://html.duckduckgo.com/html/"

        # TC01: Search Python programming
        await page.goto(url, wait_until="domcontentloaded")
        await page.screenshot(
            path="screenshots/TC01_before_search.png"
        )

        await page.locator("input[name='q']").fill(
            "Python programming"
        )
        await page.screenshot(
            path="screenshots/TC01_entered_text.png"
```

```
)

await page.locator("input[name='q']").press("Enter")
await page.wait_for_timeout(5000)

await page.screenshot(
    path="screenshots/TC01_search_results.png"
)

results = page.locator(".result")
count = await results.count()

if count > 0:
    print(f"TC01: Python search - PASS ({count} results)")
else:
    print("TC01: Python search - CHECK website response")

# TC02: Search Playwright testing
await page.goto(url, wait_until="domcontentloaded")
await page.locator("input[name='q']").fill(
    "Playwright testing"
)
await page.screenshot(
    path="screenshots/TC02_entered_text.png"
)

await page.locator("input[name='q']").press("Enter")
await page.wait_for_timeout(5000)

await page.screenshot(
    path="screenshots/TC02_search_results.png"
)

count = await page.locator(".result").count()

if count > 0:
    print(f"TC02: Playwright search - PASS ({count} results)")
else:
    print("TC02: Playwright search - CHECK website response")

# TC03: Empty search
await page.goto(url, wait_until="domcontentloaded")
await page.screenshot(
    path="screenshots/TC03_empty_search.png"
```

```
    )

    await page.locator("input[name='q']").fill("")
    await page.locator("input[name='q']").press("Enter")
    await page.wait_for_timeout(2000)

    await page.screenshot(
        path="screenshots/TC03_after_empty_search.png"
    )

    print("TC03: Empty search tested")

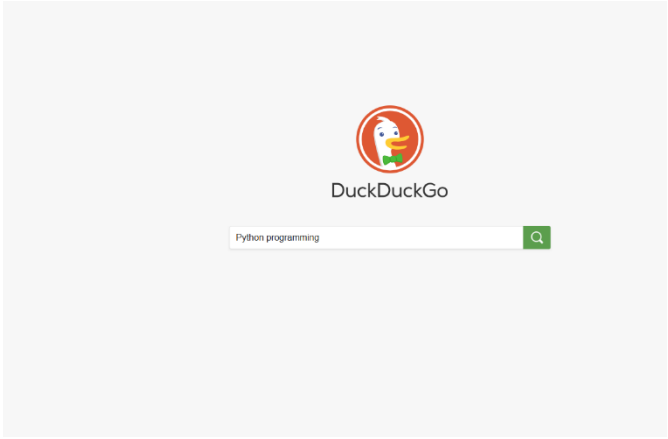
    print("Screenshots saved in the screenshots folder.")

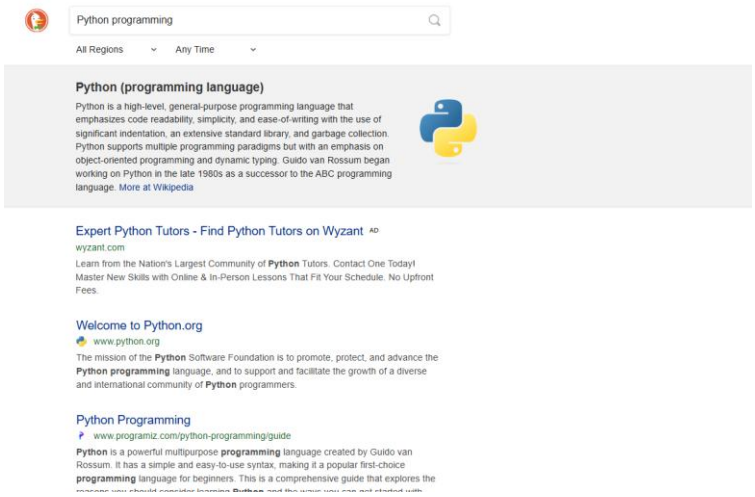
    await page.wait_for_timeout(5000)
    await browser.close()

asyncio.run(test_search())
```

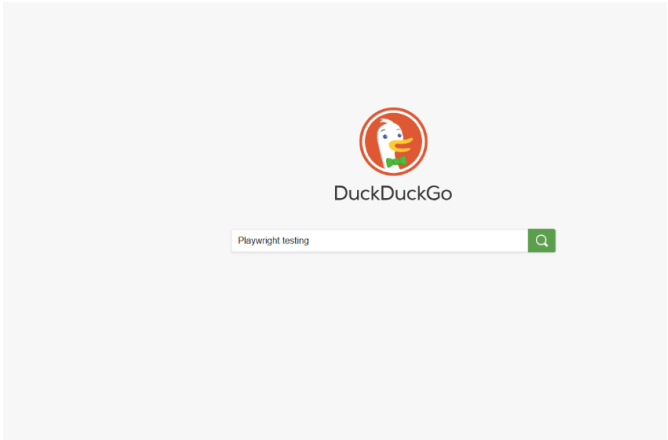
Output

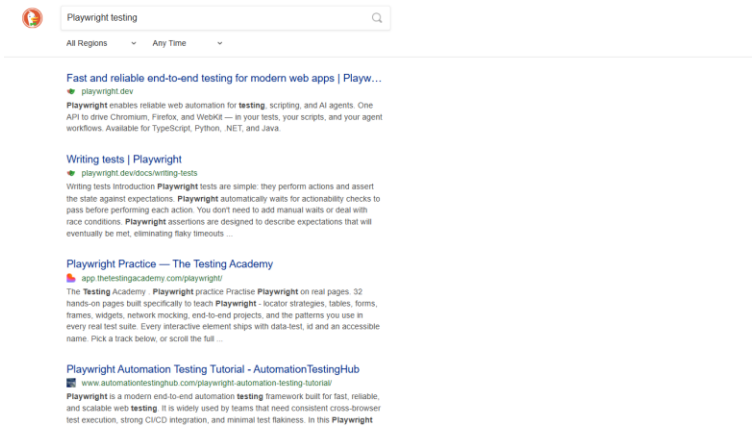
Python programming



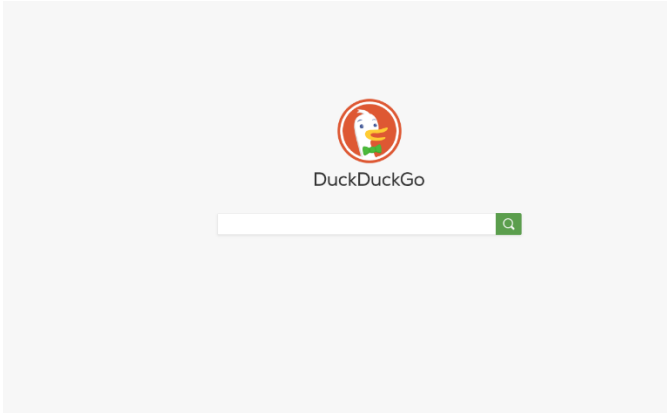


Playwright testing





Empty input



Aim/Overview of the practical: (B). Perform cross-browser testing using Playwright to ensure consistent behavior and functionality of the web application across multiple browsers.

Objective: The objective is to perform cross-browser testing using Playwright to verify that a web application functions consistently across multiple browsers, such as Chromium, Firefox, and WebKit, ensuring compatibility and a consistent user experience.

Code

```
import asyncio
import os
from playwright.async_api import async_playwright

async def test_cross_browser():
    os.makedirs("screenshots", exist_ok=True)

    async with async_playwright() as p:

        browsers = [
            ("Chromium", p.chromium),
            ("Firefox", p.firefox),
            ("WebKit", p.webkit)
        ]

        for name, browser_type in browsers:
            browser = None
            page = None

            try:
                print(f"\nTesting on {name} browser...")

                browser = await browser_type.launch(
```

```
    headless=False,
    slow_mo=1000
)

page = await browser.new_page(
    viewport={"width": 1366, "height": 768}
)

# Open login page
await page.goto(
    "https://www.saucedemo.com/",
    wait_until="domcontentloaded",
    timeout=60000
)
await page.wait_for_timeout(3000)

await page.screenshot(
    path=f"screenshots/{name}_01_login_page.png"
)

# Enter username
await page.locator("#user-name").fill("standard_user")
await page.wait_for_timeout(2000)

# Enter password
await page.locator("#password").fill("secret_sauce")
await page.wait_for_timeout(2000)

await page.screenshot(
    path=f"screenshots/{name}_02_credentials.png"
)

# Click login
await page.locator("#login-button").click()
await page.wait_for_timeout(5000)

# Verify login
heading = page.locator(".title")

if (
    await heading.count() > 0
    and await heading.inner_text() == "Products"
):
    print(f"{name}: Login test - PASS")
```

```
            else:
                print(f"{name}: Login test - FAIL")
                print("Current URL:", page.url)

            await page.screenshot(
                path=f"screenshots/{name}_03_login_result.png"
            )

            # Keep browser open to view the result
            await page.wait_for_timeout(5000)

        except Exception as e:
            print(f"{name}: Test error - {e}")

            if page:
                try:
                    await page.screenshot(
                        path=f"screenshots/{name}_error.png"
                    )
                except Exception:
                    pass

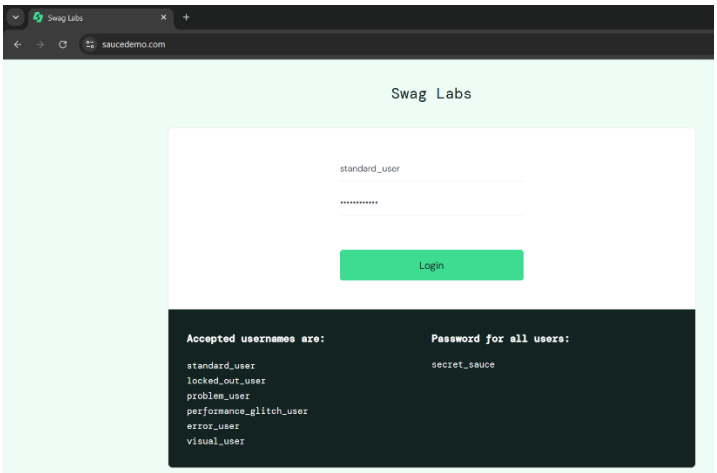
        finally:
            if browser:
                await browser.close()

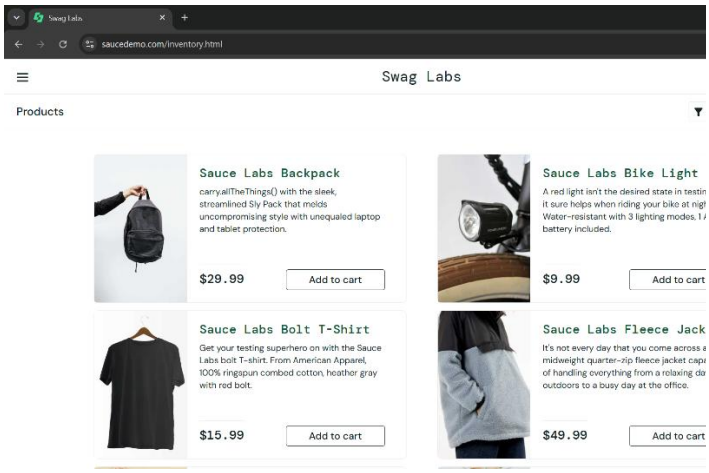
    print("\nTesting completed. Check the screenshots folder.")

asyncio.run(test_cross_browser())
```

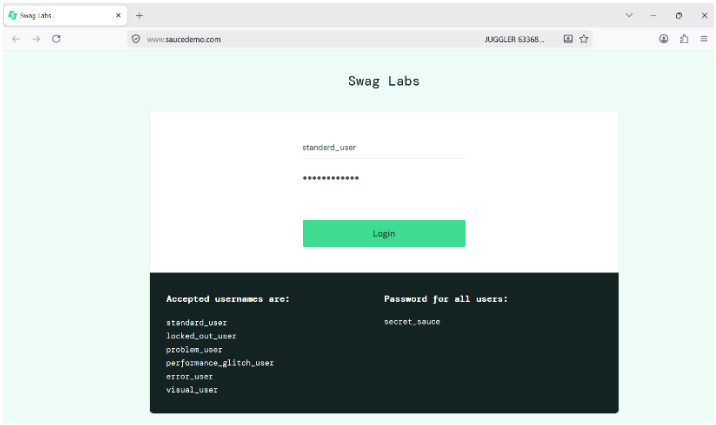
Output

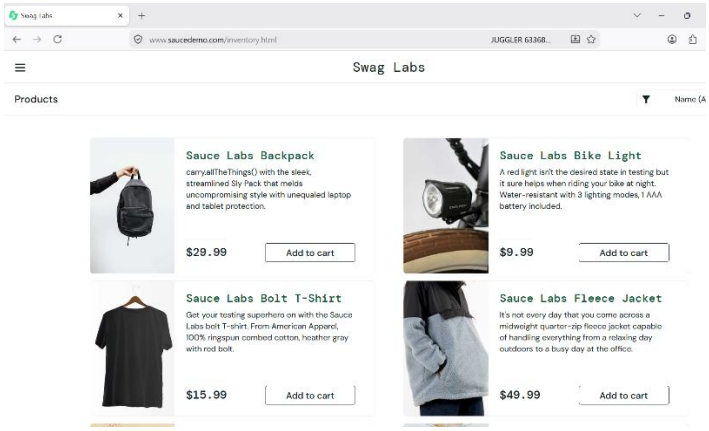
Chrome



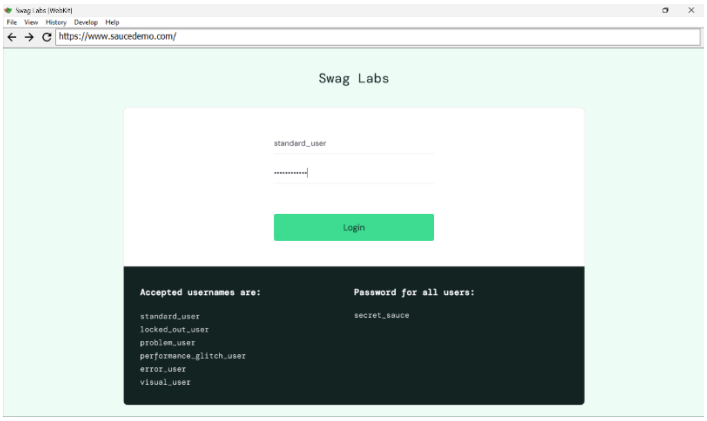


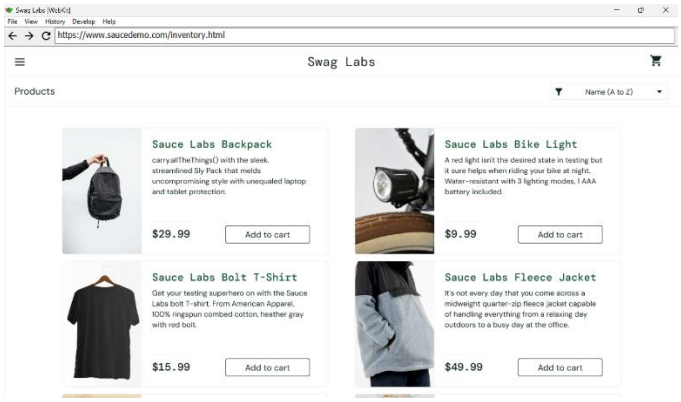
FireFox





Webkit





Aim/Overview of the practical:(C). Validate session management by automating login and logout operations, ensuring proper session creation, maintenance, and termination under various scenarios.

Objective: The objective is to validate session management using Playwright by automating login and logout operations to ensure proper session creation, maintenance, and termination, and to verify that unauthorized users cannot access protected pages after logout.

Code

```
import asyncio
import os
from playwright.async_api import async_playwright

async def test_session():
    os.makedirs("screenshots", exist_ok=True)

    async with async_playwright() as p:
        browser = await p.chromium.launch(
            headless=False,
            slow_mo=1000
```

```
)

page = await browser.new_page(
    viewport={"width": 1366, "height": 768}
)

url = "https://www.saucedemo.com/"

# TC01: Login with valid credentials
await page.goto(url)
await page.wait_for_timeout(3000)

await page.screenshot(
    path="screenshots/TC01_login_page.png"
)

await page.locator("#user-name").fill("standard_user")
await page.wait_for_timeout(2000)

await page.locator("#password").fill("secret_sauce")
await page.wait_for_timeout(2000)

await page.screenshot(
    path="screenshots/TC01_entered_credentials.png"
)

await page.locator("#login-button").click()
await page.wait_for_url("**/inventory.html", timeout=15000)
await page.wait_for_timeout(3000)

print("TC01: Login successful - Pass")

await page.screenshot(
    path="screenshots/TC01_login_success.png"
)

# TC02: Check session maintenance after refresh
await page.wait_for_timeout(3000)
await page.reload()
await page.wait_for_timeout(4000)

if "inventory.html" in page.url:
    print("TC02: Session maintained after refresh - Pass")
else:
```

```
    print("TC02: Session maintenance - Fail")

await page.screenshot(
    path="screenshots/TC02_session_refresh.png"
)

# TC03: Logout
await page.wait_for_timeout(3000)
await page.locator("#react-burger-menu-btn").click()
await page.wait_for_timeout(2000)

await page.screenshot(
    path="screenshots/TC03_logout_menu.png"
)

await page.locator("#logout_sidebar_link").click()
await page.wait_for_url("**/", timeout=15000)
await page.wait_for_timeout(3000)

if await page.locator("#login-button").is_visible():
    print("TC03: Logout completed - Pass")
else:
    print("TC03: Logout - Fail")

await page.screenshot(
    path="screenshots/TC03_logout_success.png"
)

# TC04: Try opening protected page after logout
await page.wait_for_timeout(3000)
await page.goto(
    "https://www.saucedemo.com/inventory.html"
)
await page.wait_for_timeout(4000)

if await page.locator("#login-button").is_visible():
    print("TC04: Login required after logout - Pass")
else:
    print("TC04: Verify protected page access")

await page.screenshot(
    path="screenshots/TC04_protected_page.png"
)
```

```
    # Keep the final browser page visible
    await page.wait_for_timeout(10000)
    await browser.close()

asyncio.run(test_session())
```

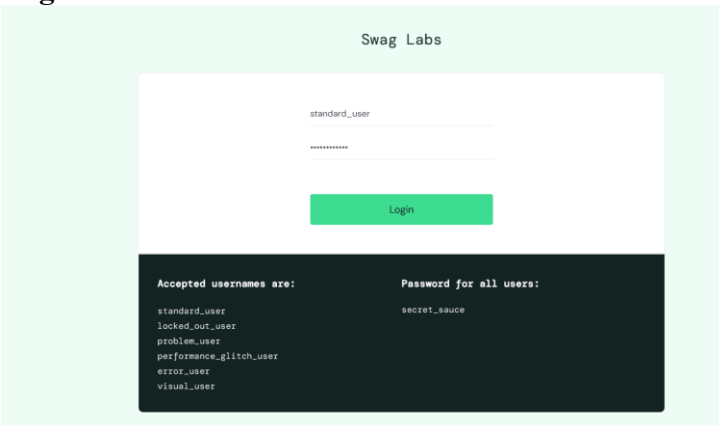
Test Case

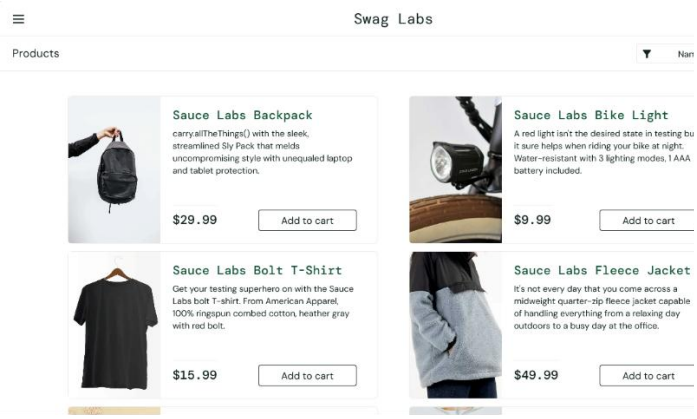
Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC01	Search using a valid keyword	Python programming	Relevant search results should be displayed.	Search results displayed for Python programming.	Pass
TC02	Search using another keyword	Playwright testing	Results related to Playwright should be displayed.	Search results displayed for Playwright testing.	Pass
TC03	Search with empty input	Blank input	The website should handle the empty search without crashing.	Website remained accessible after submitting empty input.	Pass
TC04	Login using Chromium	standard_user / secret_sauce	User should log in and see the Products page.	Products page displayed successfully in Chromium.	Pass
TC05	Login using Firefox	standard_user / secret_sauce	User should log in and see the Products page.	Products page displayed successfully in Firefox.	Pass
TC06	Login using WebKit	standard_user / secret_sauce	User should log in and see the Products page.	Products page displayed successfully in WebKit.	Pass

TC07	Login with valid credentials	standard_user / secret_sauce	User should be redirected to the Products page.	User logged in and the Products page was displayed.	Pass
TC08	Maintain session after page refresh	Refresh the Products page after login	User should remain logged in.	User remained logged in after refreshing the page.	Pass
TC09	Logout from the application	Click the Logout option	User should be redirected to the login page.	User was redirected to the login page.	Pass
TC10	Access protected page after logout	Open /inventory.html directly	User should not access the protected Products page without logging in.	Login page was displayed when the protected page was opened after logout.	Pass

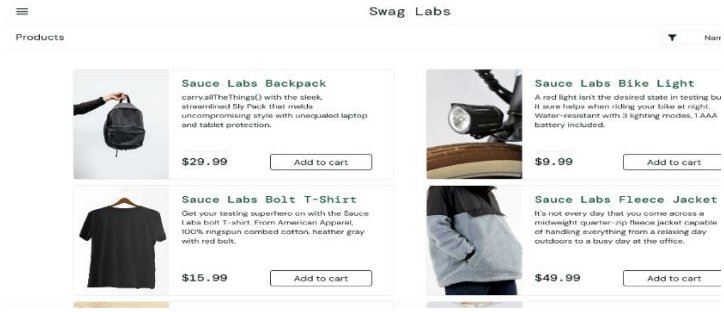
Output

Login

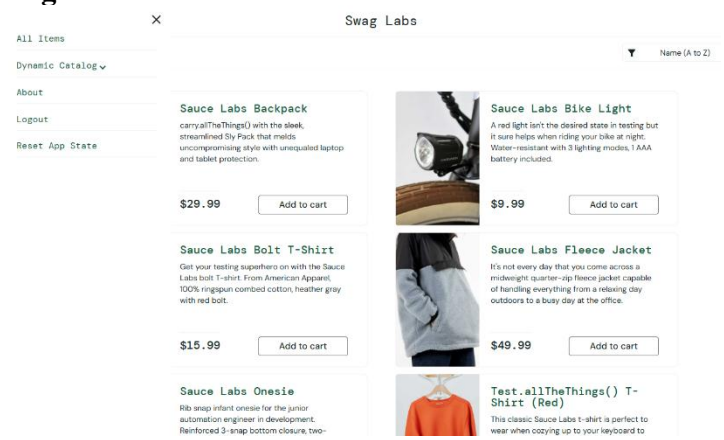


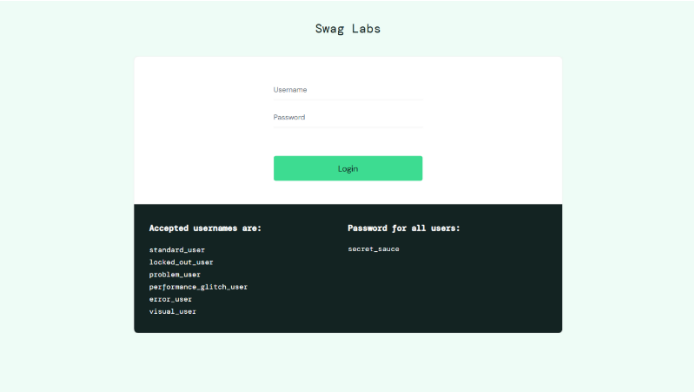


Refresh Page



Logout





Learning outcomes (What I have learnt):

1. Learned to develop and execute automated test cases using Playwright to validate search functionality.
2. Gained knowledge of verifying search results for valid, invalid, and empty inputs.
3. Understood how to perform cross-browser testing to ensure consistent application behavior across different browsers.
4. Learned to automate login and logout operations to validate session management.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Viva		10